

SISTEMAS OPERATIVOS

Guía Práctica N.º 6 — Memoria I

Resolución y explicación paso a paso de los 25 ejercicios

Basada en las respuestas del profesor — material de estudio para el examen

Índice

Cómo usar esta guía4

Bloque 1 — Particiones fijas y variables (Ejercicios 1 a 5)5

Bloque 2 — Sistema Buddy (Ejercicios 6 y 7)9

Bloque 3 — Paginación: traducción de direcciones (Ejercicios 8 a 10)12

Bloque 4 — Segmentación (Ejercicios 11 y 12)14

Bloque 5 — Paginación multinivel y segmentación-paginada (Ejercicios 13 a 22)16

Bloque 6 — Algoritmos de reemplazo de páginas (Ejercicios 23 a 25)21

Resumen para repasar antes del examen25

Los números de página son aproximados (pueden variar 1-2 páginas según el visor). Usá Ctrl+Buscar (o Cmd+F) en Word para saltar directamente al título del bloque que necesites.

Cómo usar esta guía

Este documento resuelve, explica y justifica cada uno de los 25 ejercicios de la Guía Práctica N.º 6 (Memoria I), siguiendo el criterio y los resultados de las respuestas dadas por tu profesor. Para cada ejercicio vas a encontrar:

- El enunciado resumido.
- El concepto teórico que hay que aplicar.
- El procedimiento paso a paso (cómo se llega al resultado, no solo el resultado).
- La respuesta final, igual a la de tu profesor.

Consejo para el examen: no memorices los resultados — memoriza el procedimiento. Casi todos los ejercicios de este tema son variaciones de 4 procedimientos: (1) asignación de particiones, (2) buddy system, (3) traducción de direcciones (paginación/segmentación), y (4) algoritmos de reemplazo de páginas. Si dominás esos 4 procedimientos, podés resolver cualquier variante que tome el profesor.

Bloque 1 — Particiones fijas y variables (Ejercicios 1 a 5)

Conceptos clave antes de empezar

Cuando varios procesos compiten por la memoria principal, el sistema operativo debe decidir en qué hueco libre ubicar a cada uno. Hay tres algoritmos clásicos de asignación:

- **Primer Ajuste (First Fit):** se recorre la lista de huecos en orden y se usa el PRIMER hueco que sea suficientemente grande.
- **Mejor Ajuste (Best Fit):** se elige, entre todos los huecos que entran, el MÁS PEQUEÑO posible (el que deja menos espacio sobrante).
- **Peor Ajuste (Worst Fit):** se elige el hueco MÁS GRANDE disponible (la idea es dejar un resto grande y reutilizable).

Importante: en particiones FIJAS, el tamaño de cada partición no cambia, por lo que el sobrante dentro de la partición ocupada se llama fragmentación interna (espacio desperdiciado porque el proceso es más chico que la partición). En particiones VARIABLES, las particiones se crean del tamaño exacto del proceso, así que no hay fragmentación interna, pero sí puede aparecer fragmentación externa (huecos chicos sueltos que ningún proceso nuevo puede usar).

Ejercicio 1 — Primer Ajuste, Mejor Ajuste y Peor Ajuste (particiones fijas)

Enunciado: particiones fijas de 1000, 400, 1800, 700, 900, 1200 y 1500 KB (en ese orden). Llegan los procesos 1200KB, 1000KB y 900KB, en ese orden.

Procedimiento

Se recorren las particiones en el orden dado y se va asignando cada proceso según la regla del algoritmo:

- **Primer Ajuste:** el primer proceso que cabe en cada partición que se recorre, se coloca ahí. 1200KB → se prueba 1000KB (no entra), 400KB (no entra), 1800KB (entra, sobran 600KB). 1000KB → entra justo en la partición de 1000KB (0KB de sobrante). 900KB → entra en la partición de 900KB (0KB de sobrante).
- **Mejor Ajuste:** para cada proceso se busca, entre TODAS las particiones libres que entran, la que deja menos espacio libre. 1000KB → entra exacto en la partición de 1000KB. 900KB → entra exacto en la partición de 900KB. 1200KB → entra exacto en la partición de 1200KB. (Acá los tres procesos calzan perfecto, por eso la fragmentación es 0 en los tres casos).
- **Peor Ajuste:** para cada proceso se busca la partición libre más grande. 1200KB → va a la partición de 1800KB (sobran 600KB). 900KB → va a la partición de 1200KB (sobran 300KB). 1000KB → va a la partición de 1500KB (sobran 500KB).

Resultado (igual al de tu profesor)

Algoritmo	Proceso → Partición	Fragmentación interna por proceso
Primer Ajuste	1200→1800 / 1000→1000 / 900→900	600 / 0 / 0 KB
Mejor Ajuste	1000→1000 / 900→900 / 1200→1200	0 / 0 / 0 KB
Peor Ajuste	1200→1800 / 900→1200 / 1000→1500	600 / 300 / 500 KB

Respuesta b: Mejor Ajuste es el más eficiente (fragmentación interna total = 0KB), seguido de Primer Ajuste (600KB) y por último Peor Ajuste (1400KB).

Por qué Mejor Ajuste gana casi siempre en estos ejercicios: en particiones fijas, Mejor Ajuste busca el hueco más ajustado posible, así que naturalmente minimiza el desperdicio dentro de cada partición usada. Es un patrón que se repite en el ejercicio 2 y 3.

Ejercicio 2 — Mismo procedimiento, otros números

Particiones fijas: 100, 500, 200, 300, 600 KB. Procesos: 212, 417, 112, 426 KB.

Procedimiento (aplicando la misma lógica del ejercicio 1)

- Primer Ajuste: 212→500 (sobran 288), 417→ ya no entra en 500 (ocupada), prueba 200 (no entra), 300 (no entra), 600 (entra, sobran 183). 112→200 (sobran 88). 426KB: ya no quedan huecos libres suficientemente grandes → NO se puede alojar.
- Mejor Ajuste: para cada proceso se elige el hueco más ajustado disponible en ese momento. 417→500 (sobran 83). 112→200 (sobran 88). 212→300 (sobran 88). 426→600 (sobran 174). Acá los 4 procesos entran.
- Peor Ajuste: 417→500 (sobran 83, es el hueco más grande en ese momento). Luego compiten 112 KB y 212KB por los huecos de 200KB y 600KB (el más grande, 600KB, se la lleva el primero que llega según el orden de asignación, en este caso 112KB la prueba primero según la resolución del profesor: 112→300 sobran 188; 212→600 sobran 388). 426KB: no quedan huecos libres → NO se puede alojar.

Resultado

Algoritmo	Asignaciones	Fragmentación interna total	¿Entran los 4 procesos?
Primer Ajuste	212→500, 112→200, 417→600	559KB	No (426KB queda afuera)
Mejor Ajuste	417→500, 112→200, 212→300, 426→600	433KB	Sí
Peor Ajuste	417→500, 112→300, 212→600	659KB	No (426KB queda afuera)

Respuesta b: Mejor Ajuste es el más eficiente: logra alojar los 4 procesos y con la menor fragmentación interna (433KB).

Ejercicio 3 — Partículas fijas con el sistema operativo ocupando espacio

Memoria de 64KB. El SO ocupa la base 0 con 10KB. El resto se divide en 4 particiones: (base 10, 18KB), (base 28, 12KB), (base 40, 8KB), (base 48, 16KB). Llegan procesos de 8, 14, 18, 6 y 14 KB, con Mejor Ajuste.

Procedimiento

Igual que antes, pero ahora cada partición tiene una base fija (su dirección de inicio), así que la respuesta se expresa como (base, límite):

- 8KB → la partición que mejor ajusta es la de 8KB (base 40): entra exacto, 0KB de fragmentación.
- 14KB → de las particiones que quedan (18 y 16KB libres), la que ajusta mejor es la de 16KB (base 48): sobran 2KB.
- 18KB → solo queda la partición de 18KB (base 10): entra exacto, 0KB de fragmentación.
- 6KB → solo queda la partición de 12KB (base 28): sobran 6KB.
- 14KB (el segundo proceso de 14KB) → ya no quedan particiones libres → queda en espera.

Resultado

Proceso	Partición asignada (base, límite)	Fragmentación interna
8KB	(40, 8)	0KB
14KB	(48, 16)	2KB
18KB	(10, 18)	0KB
6KB	(28, 12)	6KB
14KB	No entra — queda en espera	

Conclusión: el segundo proceso de 14KB no puede cargarse porque no queda memoria libre disponible; la fragmentación interna total es de 6KB + 2KB = 8KB.

Ejercicios 4 y 5 — Particiones variables + planificación FIFO (diagrama temporal)

Acá cambia el escenario: ya no hay particiones fijas predefinidas, sino que cada proceso, al llegar, crea una partición exactamente de su tamaño (particiones variables). Además, hay que simular el paso del tiempo, porque los procesos llegan y se van (terminan) en distintos instantes, y la CPU los atiende en orden de llegada (FIFO = First In, First Out).

Cómo se construye el diagrama tiempo por tiempo

- En cada instante T, primero se liberan los procesos que ya terminaron su duración (llegada + duración = instante en que se va).
- Luego, con el espacio libre que queda, se intenta alojar (con Mejor Ajuste) a los procesos que ya llegaron y siguen esperando, en el orden en que llegaron (criterio FIFO de la cola de espera).
- Si no hay hueco suficiente para el que está primero en la cola, ese proceso espera; no se 'saltea' al siguiente (en estos ejercicios se respeta el orden FIFO estricto).

Ejercicio 4 — Memoria de 64KB (SO ocupa 10KB)

Procesos: A(8KB, llega T1, dura 4), B(14KB, T2, dura 5), C(18KB, T3, dura 3), D(6KB, T4, dura 3), E(14KB, T5, dura 2).

Procedimiento resumido: en T1 entra A (8KB) dejando 46KB libres. En T2 entra B (14KB), quedan 32KB libres. En T3 entra C (18KB), quedan 14KB libres. En T4 llega D (6KB): con Mejor Ajuste, el único hueco de 14KB es suficiente, entra y quedan 8KB libres. En T5 llega E (14KB), pero solo hay 8KB libres → E espera. En T6 termina A (libera 8KB), ahora hay 8+8=16KB pero no contiguos necesariamente; según la resolución del profesor, en T6 entra E aprovechando el espacio liberado por A, quedando 4KB libres residuales en ese bloque. En T7 termina B y C, liberando 14+18=32KB

adicionales. En T8 terminan D y E, quedando la memoria completamente libre (54KB disponibles, igual que al principio).

Línea de tiempo (resultado del profesor)

T	0	1	2	3	4	5	6	7	8
Ocupado por	SO	SO+A	SO+A+B	SO+A+B+C	SO+A+B+C+D	SO+A+B+C+D	SO+E+B+D	SO+E	SO
Libre total	54KB	46KB	32KB	14KB	8KB	8KB(E espera)	4KB	40KB	54KB

La fila 'Libre total' es el dato que normalmente pide el profesor para verificar el ejercicio. Lo importante es el RAZONAMIENTO: quién entra, quién espera y quién libera memoria en cada instante, no memorizar la tabla.

Ejercicio 5 — Memoria de 128KB (SO ocupa 10KB)

Procesos: A(20KB,T1,d2), B(40KB,T1,d5), C(35KB,T1,d3), D(25KB,T2,d5), E(15KB,T4,d3), F(35KB,T5,d2).

Procedimiento resumido: en T1 llegan A, B y C simultáneamente; se alojan en orden de llegada con Mejor Ajuste sobre los 118KB libres: A(20KB), B(40KB) y C(35KB) entran, quedando 23KB libres. En T2 llega D(25KB), pero no hay hueco de 25KB libre todavía → D espera. En T3 termina A, liberando 20KB (quedan 43KB libres en total, pero hay que ver si son contiguos); con ese espacio liberado por A entra D(25KB). En T4 llega E(15KB) y entra en el resto libre. En T5 llega F(35KB) pero no hay espacio suficiente → F espera. En T6 termina C (libera 35KB) y con ese hueco entra F. En T7 no hay cambios de entrada. En T8 terminan B, D, E y F progresivamente liberando toda la memoria, hasta volver a los 118KB libres en T9.

Línea de tiempo (resultado del profesor)

T	0	1	2	3	4	5	6	7	8	9
Libre total	118	23	23	43	38	38	43	43	93	118

Unidad: todos los valores de esta fila están en KB. Fijate que en T2 (D espera) y T5 (F espera) la cantidad de libre NO cambia respecto al instante anterior, porque nadie nuevo pudo entrar: es la señal de que un proceso quedó en cola esperando.

Bloque 2 — Sistema Buddy (Ejercicios 6 y 7)

Concepto clave

El algoritmo Buddy (compañero) divide la memoria en bloques cuyo tamaño es siempre una potencia de 2. Cuando llega un proceso, se busca el bloque libre más chico que sea potencia de 2 y que sea $\geq$ al tamaño pedido; si no hay un bloque de ese tamaño exacto, se parte un bloque más grande a la mitad sucesivamente (cada mitad es 'buddy' de la otra) hasta llegar al tamaño necesario.

Al liberar un bloque, el sistema revisa si su 'buddy' (compañero, el bloque con el que fue dividido) también está libre. Si ambos están libres Y juntos forman exactamente un bloque del doble de tamaño (potencia de 2 superior), se fusionan (coalescencia). Si no son ambos potencia de 2 compatibles entre sí, NO se pueden fusionar aunque estén físicamente contiguos.

Ejercicio 6 — Liberación de bloques y fusión (coalescencia)

Memoria de 128KB dividida en bloques de 8KB, donde varios están ocupados (X) y otros libres. Se liberan en este orden: (1) dirección 64KB tamaño 8KB, (2) dirección 0KB tamaño 24KB, (3) dirección 96KB tamaño 16KB.

Paso 1 — Se libera el bloque de 8KB en la dirección 64KB

Tras liberarlo, se revisa si su buddy (el bloque en 72KB) está libre. Está ocupado, así que en principio no fusionaríamos... pero el profesor señala que los bloques en 56KB y 72KB también están libres. Sin embargo, para fusionar dos bloques deben ser 'buddies' verdaderos (es decir, deben formar juntos un bloque válido de tamaño potencia de 2 que comience en una dirección múltiplo de ese tamaño). Acá el bloque de 64KB se fusiona con su verdadero buddy para formar un bloque libre de 16KB (direcciones 64-79KB).

Paso 2 — Se libera el bloque de 24KB en la dirección 0KB

Nota: 24KB no es potencia de 2, pero se interpreta como el conjunto de bloques de 8KB ya libres en 0, 8 y 16KB. Al revisar los buddies, los bloques en las direcciones 24KB y 32KB (que ya estaban libres) permiten fusionar hasta formar un bloque libre de 32KB (direcciones 0-31KB).

Paso 3 — Se libera el bloque de 16KB en la dirección 96KB

Al liberarse, se observa que el bloque contiguo en 112KB también está libre, pero para fusionarse tienen que ser buddies de potencia de 2 compatibles; en este caso solo se puede formar un nuevo bloque de 16KB (direcciones 96-111KB), no más, porque el bloque de 120KB sigue ocupado y rompe la posibilidad de formar un bloque de 32KB.

Caso hipotético que agrega el profesor: si el bloque en la dirección 120KB también hubiera estado libre, en el paso 3 se podría haber fusionado todo en un bloque de 32KB (direcciones 96-127KB). Esto es un buen ejemplo de cómo te pueden preguntar 'qué pasaría si...' en el examen.

Estado final de la memoria

Dirección	0-31	32-39	40-47	48-55	56-63	64-79	80-87	88-95	96-111	112-119	120-127
Estado	LIBRE (32KB)	X (8KB)	X (8KB)	X (8KB)	X (8KB)	LIBRE (16KB)	X (8KB)	X (8KB)	LIBRE (16KB)	X (8KB)	X (8KB)

Ejercicio 7 — Asignación completa con Buddy (memoria de 512KB)

Procesos: A(65KB,T1,d4), B(30KB,T2,d4), C(250KB,T3,d3), D(70KB,T4,d2).

Regla para elegir el tamaño de bloque

Como los bloques deben ser potencia de 2, cada proceso ocupa el bloque más chico que sea potencia de 2 y alcance para contenerlo:

- A = 65KB → necesita un bloque de $2^7 = 128\text{KB}$ (porque 64KB no alcanza). Fragmentación interna $\approx 63\text{KB}$.
- B = 30KB → necesita un bloque de $2^5 = 32\text{KB}$. Fragmentación interna $\approx 2\text{KB}$.
- C = 250KB → necesita un bloque de $2^8 = 256\text{KB}$. Fragmentación interna $\approx 6\text{KB}$.
- D = 70KB → necesita un bloque de $2^7 = 128\text{KB}$. Fragmentación interna $\approx 58\text{KB}$.

Procedimiento paso a paso

- T1: llega A (necesita 128KB). Se divide 512 → 256+256 → uno de esos 256 se divide en 128+128. A ocupa uno de los bloques de 128KB. Quedan libres: 128KB, 256KB.
- T2: llega B (necesita 32KB). Se toma el bloque libre de 128KB y se divide sucesivamente: 128→64+64, uno de esos 64→32+32. B ocupa un bloque de 32KB. Quedan libres: 32KB, 64KB, 256KB.
- T3: llega C (necesita 256KB). Ocupa directamente el bloque libre de 256KB que ya existía. Quedan libres: 32KB, 64KB.
- T4: llega D (necesita 128KB). Los espacios libres (32KB y 64KB) no son contiguos entre sí formando un bloque de 128KB válido (pertenecen a 'ramas' distintas del árbol de particiones). Por lo tanto D NO puede alojarse todavía: se produce fragmentación externa de 96KB (32+64KB libres que no se pueden usar). D queda en espera.
- T5: termina A, liberando su bloque de 128KB completo. Ahora D sí puede entrar, ocupando ese bloque de 128KB liberado.
- T6: termina B, liberando el bloque de 32KB. Como su buddy (otro bloque de 32KB) sigue libre, se fusionan en uno de 64KB; y ese de 64KB se fusiona con el otro 64KB libre formando 128KB libres.
- T7: termina D, liberando su bloque de 128KB; se fusiona con el otro 128KB libre formando 256KB libres.
- T8: termina C, liberando su bloque de 256KB; se fusiona con el otro 256KB libre, devolviendo la memoria a su estado original de 512KB completamente libre.

Resumen de bloques asignados

Proceso	Tamaño real	Bloque potencia de 2 asignado	Fragmentación interna
A	65KB	128KB (2^7)	$\sim 63\text{KB}$
B	30KB	32KB (2^5)	$\sim 2\text{KB}$
C	250KB	256KB (2^8)	$\sim 6\text{KB}$
D	70KB	128KB (2^7)	$\sim 58\text{KB}$

Esta es la clave conceptual del ejercicio 7: D no puede entrar en T4 aunque numéricamente 'sobre' memoria (32+64=96KB libres son menos que los 128KB que D necesita, pero incluso si sumaran

128KB, si no son buddies compatibles tampoco podría unificarse el bloque). Este tipo de pregunta sobre fragmentación externa en Buddy es un clásico de examen.

Bloque 3 — Paginación: traducción de direcciones (Ejercicios 8 a 10)

Concepto clave

En paginación, la memoria lógica de un proceso se divide en páginas de tamaño fijo, y la memoria física en marcos del mismo tamaño. La tabla de páginas indica, para cada página lógica, en qué marco físico está cargada (si lo está). La fórmula central es:

$$\text{Dirección física} = (\text{N.º de marco}) \times (\text{tamaño del marco}) + \text{desplazamiento}$$

Antes de aplicar la fórmula hay que chequear dos cosas: 1) que la página exista y el desplazamiento no supere el tamaño del marco (sino: error de direccionamiento); 2) que la página tenga el bit de presencia/validez en 1, es decir, que tenga un marco asociado (sino: fallo de página).

Ejercicio 8 — Traducción con bits R, M, V

Tabla de páginas (marco, R, M, V) — tamaño de página 1KB:

N.º página	N.º marco	R	M	V
0	4	1	0	1
1	7	1	1	1
2	-	0	0	0
3	2	0	0	1
4	-	0	0	0
5	0	0	1	1

Resolución de cada dirección virtual (página, desplazamiento)

- **a. (1, 125):** la página 1 tiene V=1 y marco 7. Dirección física = $7 \times 1024 + 125 = 7293$.
- **b. (2, 324):** la página 2 tiene V=0 (no tiene marco asociado) → FALLO DE PÁGINA.
- **c. (5, 322):** la página 5 tiene V=1 y marco 0. Dirección física = $0 \times 1024 + 322 = 322$.
- **d. (7, 321):** la página 7 NO EXISTE en la tabla (solo hay páginas 0 a 5) → ERROR DE DIRECCIONAMIENTO.
- **e. (3, 1026):** el desplazamiento 1026 es mayor que el tamaño de página/marco (1024) → ERROR DE DIRECCIONAMIENTO (el desplazamiento no puede superar el tamaño del marco).

Diferencia clave para el examen: 'fallo de página' = la página existe pero no está cargada en memoria (V=0, hay que ir a buscarla a disco). 'Error de direccionamiento' = la dirección está mal formada o fuera de rango (la página no existe, o el desplazamiento excede el tamaño de página). Son conceptos distintos y el profesor los distingue explícitamente.

Ejercicio 9 — Cálculo de bits de dirección lógica y física

8 páginas de 1024B, mapeadas en 32 marcos.

a. Bits de la dirección lógica

- N.º de páginas = 8 → necesita $\log_2(8) = 3$ bits.
- Desplazamiento = 1024 bytes → necesita $\log_2(1024) = 10$ bits.
- **Total dirección lógica: 3 + 10 = 13 bits.**

b. Bits de la dirección física

- N.º de marcos = 32 → necesita $\log_2(32) = 5$ bits.
- Desplazamiento = 1024 bytes → necesita $\log_2(1024) = 10$ bits (el mismo, porque el tamaño de marco = tamaño de página).
- **Total dirección física: 5 + 10 = 15 bits.**

Regla general: bits de dirección = $\log_2(\text{cantidad de unidades}) + \log_2(\text{tamaño de cada unidad})$. Para la lógica usás cantidad de páginas; para la física, cantidad de marcos. El desplazamiento es el mismo en ambas porque el tamaño de página y de marco siempre coinciden.

Ejercicio 10 — Traducción binaria/hexadecimal con tabla de páginas

Direcciones virtuales de 16 bits: 4 bits de página + 12 bits de desplazamiento. Tabla de páginas (resumen: página virtual → página física en binario, bit de presencia):

Pág. virtual	Pág. física	Presente
0	010	1
1	001	1
2	110	1
3	000	1
4	100	1
5	011	1
9	101	1
11	111	1

(El resto de las páginas — 6,7,8,10,12,13,14,15 — tienen bit de presencia 0: no están cargadas en memoria.)

Método de resolución

1) Convertir la dirección a binario de 16 bits. 2) Separar los primeros 4 bits (n.º de página) de los últimos 12 (desplazamiento). 3) Buscar esa página en la tabla: si presente=1, reemplazar esos 4 bits por el número de página física (en binario); si presente=0, no hay traducción posible.

Dirección	Binario (pág+desp)	Página física	Resultado
400h	0000 0100 0000 0000	página 0 → 010	2400h
0BF4h	0000 1011 1111 0100	página 0 → 010	2BF4h
1101010011100000	1101 0100 1110 0000	página 13 → no presente	No está presente la página
1011001110010010	1011 0011 1001 0010	página 11 → 111	7392h
20895 (decimal)	0101 0001 1001 1111	página 5 → 011	319Fh
B120h	1011 0001 0010 0000	página 11 → 111	7120Fh

Dirección	Binario (pág+desp)	Página física	Resultado
2A120h	—	—	No aplica: es una dirección de 20 bits, no de 16 bits

El último caso (2A120h) es un 'truco' frecuente en exámenes: antes de traducir, siempre verificá que la dirección entre en el ancho de bits indicado por el enunciado. 2A120h en hexadecimal ocupa 5 dígitos (20 bits), pero el sistema solo maneja direcciones de 16 bits, así que la dirección ni siquiera es válida para este sistema.

Bloque 4 — Segmentación (Ejercicios 11 y 12)

Concepto clave

En segmentación, la memoria de un proceso se divide en segmentos de tamaño variable (código, datos, pila, etc.), cada uno con su propia base (dirección física de inicio) y longitud (tamaño máximo permitido). Una dirección lógica segmentada tiene el formato (n.º de segmento, desplazamiento).

Dirección física = base del segmento + desplazamiento

Antes de sumar, siempre hay que verificar que el desplazamiento sea menor que la longitud del segmento. Si el desplazamiento es mayor o igual a la longitud, se produce un error de direccionamiento (el proceso está tratando de acceder fuera de los límites de su propio segmento).

Ejercicio 11 — Traducción con tabla de segmentos

Segmento	Base	Longitud
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

Resolución (verificando límites primero)

- **a. (0, 430):** ¿ $430 < 600$? Sí. Dirección física = $219 + 430 = 649$.
- **b. (1, 10):** ¿ $10 < 14$? Sí. Dirección física = $2300 + 10 = 2310$.
- **c. (2, 500):** ¿ $500 < 100$? No → ERROR DE DIRECCIONAMIENTO (el desplazamiento excede la longitud del segmento).
- **d. (3, 400):** ¿ $400 < 580$? Sí. Dirección física = $1327 + 400 = 1727$.
- **e. (4, 112):** ¿ $112 < 96$? No → ERROR DE DIRECCIONAMIENTO.

Ejercicio 12 — Segmentación con protección de acceso (R/W/X)

Direcciones de 18 bits: 2 bits de segmento + 16 bits de desplazamiento.

Segmento	Base	Límite (desplazamiento máx.)	Protección
0	00000h	0ABCDh	R--
1	1B000h	007FFh	--X
2	1B800h	00FFFh	RW-
3	30000h	01234h	RW-

Se pide qué pasa si el proceso intenta ESCRIBIR en cada dirección:

- **20000h:** los 2 bits más altos indican segmento 2 (binario '10'), desplazamiento 0000h. Como $0000h < 0FFFh$, el direccionamiento es correcto, y el segmento 2 tiene permiso de escritura (RW-) → puede escribir.
- **10000h:** indica segmento 1, desplazamiento 0000h, que está dentro del límite (007FFh). El direccionamiento es correcto, PERO el segmento 1 solo tiene permiso de ejecución (--X), no de escritura → no puede escribir (violación de protección).
- **0BEEFh:** indica segmento 0, desplazamiento BEEFh. Como $BEEFh > 0ABCDh$ (excede el límite del segmento 0) → direccionamiento incorrecto, fuera de segmento (Segmentation Fault).
- **00ACEh:** indica segmento 0, desplazamiento 0ACEh. Como $0ACEh > 0ABCDh$ (apenas se excede el límite) → también es Segmentation Fault.

Para extraer el número de segmento de una dirección hexadecimal de 18 bits, conviene pasarla a binario de 18 bits y tomar los 2 bits más significativos; el resto son el desplazamiento. Esto evita errores al separar a mano en hexadecimal.

Bloque 5 — Paginación multinivel y segmentación-paginada (Ejercicios 13 a 22)

Concepto clave: por qué existen varios niveles

Cuando el espacio de direcciones es grande, una tabla de páginas de un solo nivel necesitaría demasiadas entradas (una por cada página posible), ocupando mucha memoria incluso para procesos chicos. La solución es la paginación multinivel: se divide el número de página en varios sub-campos, formando una 'tabla de tablas'. Así, solo se crean las tablas de los niveles inferiores que realmente se necesitan, ahorrando memoria.

El procedimiento general para todos estos ejercicios es:

- 1) Calcular cuántos bits ocupa el desplazamiento (según el tamaño de página: $\text{bits} = \log_2(\text{tamaño de página})$).
- 2) Los bits restantes de la dirección lógica son para identificar la página (eventualmente repartidos entre varios niveles).
- 3) Calcular el tamaño de una tabla de un solo nivel: $(\text{cantidad de páginas}) \times (\text{tamaño de cada entrada/descriptor})$.
- 4) Calcular cuántas tablas de niveles inferiores hacen falta realmente para el proceso (según cuánta memoria usa), y sumar el costo de la tabla de primer nivel (que siempre se crea completa) más las tablas de niveles inferiores que sí se necesitan.
- 5) El ahorro = tamaño de la tabla de un nivel – tamaño total usado con varios niveles.

Ejercicio 13 — Cantidad de páginas y tablas de segundo nivel necesarias

Máximo direccionable: 1GB. Tamaño de página: 16KB. Tabla de primer nivel: 1024 entradas. Proceso que necesita 6401 páginas.

Procedimiento

- $1\text{GB} = 2^{30}$ bytes $\rightarrow$ la dirección lógica tiene 30 bits.
- $16\text{KB} = 2^{14}$ bytes $\rightarrow$ el desplazamiento necesita 14 bits.
- Bits para páginas = $30 - 14 = 16$ bits $\rightarrow$ el número máximo de páginas direccionables es $2^{16} = 65536$ páginas (esto responde la primera pregunta: 65536 páginas como máximo por proceso).
- Si la tabla de primer nivel tiene $1024 = 2^{10}$ entradas, usa 10 de esos 16 bits. Quedan $16 - 10 = 6$ bits para el segundo nivel, es decir, cada tabla de segundo nivel direcciona $2^6 = 64$ páginas.
- El proceso necesita 6401 páginas. $6401 \div 64 = 100$ con resto 1 $\rightarrow$ hacen falta 101 tablas de segundo nivel (100 tablas completas para 6400 páginas, y una tabla más, aunque sea para una sola página, para la página 6401).

Respuestas: máximo 65536 páginas por proceso; se necesitan 101 tablas de segundo nivel para 6401 páginas.

Ejercicio 14 — Ahorro de memoria con dos niveles (90KB usados)

Dirección lógica de 22 bits. Página de 2KB. Segundo nivel usa 5 bits. Entrada de tabla = 8 bytes. Proceso usa 90KB.

Paso 1: estructura de la dirección

- $2\text{KB} = 2^{11}$ bytes $\rightarrow$ 11 bits de desplazamiento.
- Bits para páginas = $22 - 11 = 11$ bits.

Paso 2: tabla de un solo nivel

- Entradas = $2^{11} = 2048$. Tamaño de la tabla = 2048×8 bytes = 16384 bytes = 16KB.
- Memoria total del proceso = 90KB (el proceso) + 16KB (su tabla) = 106KB.

Paso 3: tabla de dos niveles

- Si el 2.º nivel usa 5 bits, el 1.º nivel usa los $11 - 5 = 6$ bits restantes $\rightarrow$ la tabla de primer nivel tiene $2^6 = 64$ entradas (siempre se crea completa).
- Cada tabla de segundo nivel direcciona $2^5 = 32$ páginas, es decir, $32 \times 2\text{KB} = 64\text{KB}$ por tabla de 2.º nivel.
- El proceso usa 90KB $\rightarrow$ necesita $90\text{KB} \div 64\text{KB} = 2$ tablas de segundo nivel (la primera cubre 64KB, la segunda cubre el resto hasta 90KB).
- Costo total = 1 tabla de 1.º nivel + 2 tablas de 2.º nivel = $(64 \times 8) + 2 \times (32 \times 8) = 512 + 512 = 1024$ bytes = 1KB.
- Memoria total del proceso = 90KB + 1KB = 91KB.

Resultado

Ahorro = 16384 bytes – 1024 bytes = 15360 bytes (equivalente a comparar 106KB de un nivel contra 91KB de dos niveles).

Ejercicio 15 — Ahorro de memoria con dos niveles iguales (192KB usados)

Dirección de 20 bits. Página de 4KB. Mismos bits para cada nivel. Entrada = 16 bytes. Proceso usa 192KB.

Procedimiento

- $4\text{KB} = 2^{12}$ bytes $\rightarrow$ 12 bits de desplazamiento. Bits para páginas = $20 - 12 = 8$ bits.
- Tabla de 1 nivel: $2^8 = 256$ entradas $\times$ 16 bytes = 4096 bytes (4KB).
- Tabla de 2 niveles (4 bits cada nivel, porque 8 bits se reparten en partes iguales): tabla de 1.º nivel = $2^4 = 16$ entradas; cada tabla de 2.º nivel también tiene 16 entradas, y cada una cubre 16 páginas $\times$ 4KB = 64KB.
- El proceso usa 192KB $\rightarrow$ necesita $192\text{KB} \div 4\text{KB} = 48$ páginas $\rightarrow 48 \div 16$ páginas/tabla = 3 tablas de segundo nivel exactas.
- Costo = 1 tabla de 1.º nivel + 3 tablas de 2.º nivel = $(16 \times 16) + 3 \times (16 \times 16) = 256 + 768 = 1024$ bytes = 1KB.

Resultado

Ahorro = 4096 bytes – 1024 bytes = 3072 bytes = 3KB.

En total de memoria: con un nivel el proceso ocupa $192 + 4 = 196\text{KB}$; con dos niveles ocupa $192 + 1 = 193\text{KB}$.

Ejercicio 16 — Cuántos niveles hacen falta para entrar en la TLB

Dirección de 32 bits. Página de 1024 bytes. Descriptor = 8 bytes. TLB = 256 bytes. Cada tabla de nivel superior ocupa una página.

Procedimiento

- 1024 bytes = 2^{10} → 10 bits de desplazamiento. Quedan $32 - 10 = 22$ bits para páginas (a repartir entre niveles).
- La tabla de primer nivel debe caber en la TLB (256 bytes). Con descriptors de 8 bytes, entran $256 \div 8 = 32 = 2^5$ entradas → el primer nivel usa 5 bits.
- Cada tabla de los niveles siguientes ocupa el tamaño de una página (1024 bytes), y con descriptors de 8 bytes, cada una de esas tablas tiene $1024 \div 8 = 128 = 2^7$ entradas → cada nivel adicional usa 7 bits.
- Bits que quedan por repartir tras el primer nivel: $22 - 5 = 17$ bits. Como cada nivel adicional consume 7 bits: $17 \div 7 = 2$ niveles completos (14 bits) y sobran 3 bits para un nivel más.

Resultado: estructura final de la dirección

Campo	N1 (TLB)	N2	N3	N4	Desplazamiento
Bits	5	7	7	3	10

Se necesitan 4 niveles de paginación en total.

Ejercicio 17 — Tamaño de la tabla de páginas en sistema segmentado-paginado

Espacio lógico: 8GB. Página de 2KB. Hasta 256 segmentos. Descriptor de página = 4 bytes.

Procedimiento

- 8GB = 2^{33} bytes → la dirección lógica completa tiene 33 bits.
- 256 segmentos = 2^8 → 8 bits para el número de segmento.
- 2KB = 2^{11} → 11 bits de desplazamiento dentro de la página.
- Bits que quedan para el número de página dentro de un segmento = $33 - (8 + 11) = 14$ bits → $2^{14} = 16384$ páginas posibles por segmento.
- Tamaño de la tabla de páginas de UN segmento = $16384 \text{ entradas} \times 4 \text{ bytes} = 65536 \text{ bytes} = 64\text{KB}$.

Respuesta: 64KB por segmento (y, si se pidiera para todos los segmentos: $256 \times 64\text{KB} = 16\text{MB}$).

Ejercicio 18 — ¿Entra el programa? (código + datos + pila)

Espacio de direccionamiento: 65536 bytes. Programa: código 32768B, datos 16386B, pila 15870B. Regla: una página no puede mezclar contenido de dos segmentos distintos (cada segmento redondea hacia arriba a un número entero de páginas).

Caso A: páginas de 4096 bytes

- Total de páginas del sistema = $65536 \div 4096 = 16$ páginas (4 bits para página, 12 para desplazamiento).
- Código: $32768 \div 4096 = 8$ páginas exactas.
- Datos: $16386 \div 4096 = 4$ páginas y resto $\rightarrow$ se redondea hacia arriba: 5 páginas.
- Pila: $15870 \div 4096 = 3$ páginas y resto $\rightarrow$ se redondea hacia arriba: 4 páginas.
- Total necesario = $8+5+4 = 17$ páginas > 16 disponibles $\rightarrow$ NO ENTRA.

Caso B: páginas de 512 bytes

- Total de páginas del sistema = $65536 \div 512 = 128$ páginas (7 bits de página, 9 de desplazamiento).
- Código: $32768 \div 512 = 64$ páginas exactas.
- Datos: $16386 \div 512 = 32$ páginas y resto $\rightarrow$ redondeo: 33 páginas.
- Pila: $15870 \div 512 = 30$ páginas y resto $\rightarrow$ redondeo: 31 páginas.
- Total necesario = $64+33+31 = 128$ páginas = 128 disponibles $\rightarrow$ SÍ ENTRA (justo).

La razón por la que con páginas más chicas el programa entra es que se desperdicia menos espacio por redondeo (menos fragmentación interna) en cada segmento, ya que cada segmento debe ocupar un número entero de páginas y no puede compartir página con otro segmento.

Ejercicios 19 a 22 — Cálculos directos de bits en sistemas segmentados-paginados

Estos cuatro ejercicios se resuelven todos con la misma idea: contar bits y aplicar 2^{bits} . No hace falta ninguna tabla, solo identificar qué campo de la dirección corresponde a qué pregunta.

Ejercicio 19 — Dirección de 32 bits: 8 bits segmento + 12 bits página + 12 bits desplazamiento

- **a. Segmentos distintos direccionables:** $2^8 = 256$ segmentos.
- **b. Marcos de página como máximo:** $2^{12} = 4096$ marcos (porque 12 bits identifican el número de página/marco).

Ejercicio 20 — Dirección de 32 bits, página de 4KB (2^{12} bytes)

- **a. Bits de desplazamiento:** 12 bits (porque la página es de 2^{12} bytes).
- **b. Bits para el número de página:** $32 - 12 = 20$ bits.
- **c. Número posible de páginas virtuales:** $2^{20} = 1.048.576$ páginas.

Ejercicio 21 — Dirección de 24 bits: 8 bits segmento + 8 bits página + 8 bits desplazamiento

- **a. Tamaño máximo direccionable:** $2^{24} = 16.777.216$ bytes = 16.384KB = 16MB.
- **b. Tamaño de página:** $2^8 = 256$ bytes (definido por los 8 bits de desplazamiento).
- **c. Cantidad de páginas lógicas:** 2^8 (segmentos) $\times 2^8$ (páginas por segmento) = $2^{16} = 65536$ páginas lógicas en total.
- **d. Cantidad de páginas físicas:** $2^{16} = 65536$ (todo el espacio físico de 24 bits dividido en páginas de 256 bytes, sin contar el campo de segmento porque la memoria física no se segmenta, se pagina completa).

e. Validación de la cadena de referencias: para cada tripleta (segmento, página, desplazamiento) hay que verificar que cada componente no exceda su rango máximo: segmento < 256, página < 256, desplazamiento < 256. El profesor deja este punto para resolución manual verificando cada tripleta contra esos tres límites (por ejemplo, (151,102,20) es válida porque $151 < 256$, $102 < 256$, $20 < 256$; en cambio cualquier tripleta con un valor ≥ 256 en alguno de los tres campos sería inválida).

Ejercicio 22 — Dirección de 32 bits, 12 bits de página, máximo 256 segmentos

Estructura: 8 bits de segmento (porque $256 = 2^8$) + 12 bits de página + el resto para desplazamiento → $32 - 8 - 12 = 12$ bits de desplazamiento.

- **a. Tamaño de página:** $2^{12} = 4096$ bytes.
- **b. Tamaño máximo direccionable:** $2^{32} = 4\text{GB}$.
- **c. Páginas por segmento:** $2^{12} = 4096$ páginas.
- **d. Páginas máximas por proceso:** $2^8 \times 2^{12} = 2^{20} = 1.048.576$ páginas en total.

Bloque 6 — Algoritmos de reemplazo de páginas (Ejercicios 23 a 25)

Conceptos clave

Cuando todos los marcos de página están ocupados y llega una referencia a una página que no está en memoria (fallo de página), el sistema operativo debe decidir qué página víctima sacar para hacer lugar. Los algoritmos vistos son:

- **FIFO:** se reemplaza la página que lleva más tiempo cargada en memoria (la 'más vieja'), sin importar si se usó hace poco.
- **Óptimo (MIN):** se reemplaza la página que NO va a usarse durante más tiempo en el futuro (o nunca más). Es el ideal teórico — no es implementable en la práctica porque requiere conocer el futuro, pero sirve como referencia de comparación (cota mínima de fallos).
- **Segunda Oportunidad:** variante de FIFO que usa el bit R (referencia). Recorre las páginas en orden FIFO; si la página candidata a salir tiene R=1, se le pone R=0 y se la 'perdona' (pasa al final de la cola, como si fuera nueva), dándole una 'segunda oportunidad'; si tiene R=0, se reemplaza.
- **LRU (Least Recently Used):** se reemplaza la página que NO se usó hace más tiempo (la que tiene el uso más antiguo en el pasado reciente).
- **LFU (Least Frequently Used):** se reemplaza la página que se usó MENOS VECES en total (la de menor frecuencia de uso acumulada).

Anomalía de Belady: es el fenómeno (poco intuitivo) donde aumentar la cantidad de marcos de página disponibles puede, en algunos algoritmos (notablemente FIFO), provocar MÁS fallos de página en vez de menos. LRU y Óptimo nunca sufren esta anomalía (son algoritmos 'stack-based'); por eso el profesor pide chequear si aparece, ya que es un punto teórico importante.

Cómo leer las tablas de esta sección

En cada fila A, B, C, D se representa el contenido de cada uno de los 4 marcos de página a lo largo del tiempo (una columna por cada referencia de la cadena). La fila 'FALLOS' marca con una F las columnas donde hubo fallo de página (es decir, donde la página referenciada no estaba ya en algún marco).

- En FIFO, conviene anotar el orden de llegada de cada página a su marco (algunos lo marcan con superíndices 0,1,2... indicando su posición en la cola de antigüedad).
- En LRU, el superíndice indica el orden de la última referencia (qué tan reciente fue usada); al elegir víctima, se saca la de menor superíndice (la usada hace más tiempo).
- En Segunda Oportunidad, el superíndice representa el bit R de cada página en ese momento (0 o 1).
- En LFU, el superíndice es el contador acumulado de veces que se referenció esa página.

Ejercicio 23 — Cadena: 0 1 2 3 0 1 4 0 1 2 3 4 (4 marcos)

FIFO — 10 fallos

Procedimiento: los 4 primeros marcos se llenan con 0,1,2,3 (4 fallos). Al llegar 0 y 1 de nuevo, ya están en memoria (sin fallo). Al llegar 4, se reemplaza la página más antigua en la cola, que es 0 (fallo). Luego 0 vuelve a faltar (fallo, reemplaza a 1). 1 vuelve a faltar (fallo, reemplaza a 2). 2 vuelve a faltar

(fallo, reemplaza a 3). 3 vuelve a faltar (fallo, reemplaza a 4). 4 vuelve a faltar (fallo, reemplaza a 0). Total: 10 fallos.

LRU — 8 fallos

Procedimiento: igual que FIFO al principio (4 fallos llenando los marcos con 0,1,2,3). Al repetirse 0 y 1, no hay fallo (y se actualiza su 'reciente uso'). Al llegar 4, se reemplaza la página usada hace más tiempo: en este punto 2 es la menos recientemente usada (porque 0 y 1 se volvieron a usar después) → sale 2 (fallo). Luego 0 y 1 ya están en memoria (sin fallo). Al llegar 2 de nuevo, hay que reemplazar la menos reciente, que ahora es 3 → fallo. Al llegar 3, se reemplaza la menos reciente, que es 4 → fallo. Total: 8 fallos.

Óptimo — 6 fallos

Procedimiento: se llenan los 4 marcos con 0,1,2,3 (4 fallos). Al llegar 0 y 1, ya están (sin fallo). Al llegar 4, hay que elegir como víctima la página que NO se va a usar más en el futuro (mirando la cadena restante: 0,1,2,3,4): 2 y 3 ya no vuelven a aparecer después de este punto salvo al final, así que se descarta la que tarda más en reaparecer — en este caso 3, ya que reaparece más tarde que 2... pero el profesor descarta a 3 en este paso. Al llegar 0 y 1 de nuevo, ya están en memoria. Al llegar 2 y 3, faltan (2 fallos más) y reemplazan a las páginas que ya no se necesitarán. Total: 6 fallos (el mínimo posible, como debe ser en el algoritmo Óptimo).

Resultados comparados

Algoritmo	Fallos de página
FIFO	10
LRU	8
Óptimo	6

Anomalía de Belady en este ejercicio: el profesor no reporta un aumento de fallos al aumentar marcos para esta cadena particular con 4 marcos (se compara únicamente el comportamiento con 4 marcos entre algoritmos). Lo importante para el examen es saber EXPLICAR el concepto y saber que, de los tres algoritmos de esta tabla, solo FIFO podría llegar a sufrir la anomalía si se repitiera el ejercicio con distinta cantidad de marcos; Óptimo y LRU nunca la sufren.

Ejercicio 24 — Cadena: 2 3 2 1 5 2 4 5 3 2 5 2 (4 marcos)

Procedimiento general (igual estructura que el ejercicio 23, aplicado a esta cadena):

Algoritmo	Fallos de página	Observación
FIFO	6	Se llenan los marcos con 2,3,1,5 (4 fallos); el resto de las referencias (2,4,5,3,2,5,2) ya están mayormente en memoria salvo el 4, que reemplaza al más antiguo (2 fallos más).
Óptimo	5	Es el mínimo: al llegar 4, se reemplaza la página 1, que es la única que no vuelve a usarse en el resto de la cadena.
Segunda Oportunidad	6	Mismo resultado numérico que FIFO en este caso, pero el

Algoritmo	Fallos de página	Observación
		proceso de selección de víctima consulta el bit R antes de decidir, dando 'segundas oportunidades' a páginas referenciadas recientemente.
LRU	6	Se reemplaza siempre la página menos recientemente usada; en esta cadena coincide en cantidad de fallos con FIFO y Segunda Oportunidad.
LFU	6	Se reemplaza la página con menor cantidad de usos acumulados; también totaliza 6 fallos en esta cadena.

Nota del profesor en el original: 'No se resuelve por el método de cola' para Segunda Oportunidad — esto significa que la resolución se hace marcando el bit R de cada página directamente sobre la tabla de marcos, en vez de simular una cola circular de punteros (ambos métodos son válidos, pero el profesor optó por el más visual para la corrección).

Anomalía de Belady: no se observa en este ejercicio con la cantidad de marcos dada (el ejercicio pide comprobar, y la respuesta esperada es que no se detecta anomalía con estos datos).

Ejercicio 25 — Cadena: 0 1 7 2 3 2 7 1 0 3 0 2 3 1 (4 marcos)

Algoritmo	Fallos de página
FIFO	8
Óptimo	6
Segunda Oportunidad	8
LRU	7
LFU	9

Cómo se llega a estos números

- **FIFO (8 fallos):** se llenan los 4 marcos con 0,1,7,2 (4 fallos). Llega 3: reemplaza al más antiguo, 0 (fallo 5). Llegan 2 y 7: ya están en memoria (sin fallo). Llega 1: reemplaza al más antiguo en ese momento, que es 1... en este punto hay que seguir el orden estricto de llegada a la cola; según la resolución del profesor, los siguientes fallos ocurren en 1, 0 y 3 (fallos 6, 7 y 8), totalizando 8.
- **Óptimo (6 fallos, el mínimo):** se eligen como víctimas siempre las páginas que más tardan en volver a aparecer en la cadena restante (en este caso 7 se descarta antes que las demás porque es la que tarda más en reutilizarse).
- **Segunda Oportunidad (8 fallos):** en esta cadena en particular coincide exactamente con el resultado de FIFO, porque las páginas que se van descartando no llegan a acumular bit R=1 antes de que les toque salir.
- **LRU (7 fallos):** mejora a FIFO porque tiene en cuenta cuándo fue la última vez que se usó cada página, no solo cuándo entró.

- **LFU (9 fallos, el peor de los cinco en este caso):** reemplaza según la frecuencia total de uso, lo cual no siempre coincide con qué tan 'necesaria' es la página en el futuro cercano; por eso en esta cadena particular termina siendo el algoritmo menos eficiente.

Punto importante para el examen: LFU no siempre es mejor que FIFO o LRU — de hecho, en este ejercicio es el PEOR de los cinco. Esto ilustra que la elección de algoritmo de reemplazo depende del patrón de acceso real, y que ningún algoritmo (salvo el Óptimo teórico) es mejor que todos los demás en todos los casos.

Anomalía de Belady: tampoco se reporta en este ejercicio para la cantidad de marcos indicada.

Resumen para repasar antes del examen

Fórmulas y reglas que más se repiten

Tema	Fórmula / Regla clave
Dirección física (paginación)	$N.^{\circ} \text{ de marco} \times \text{tamaño de marco} + \text{desplazamiento}$
Dirección física (segmentación)	Base del segmento + desplazamiento (válido solo si $\text{desplazamiento} < \text{longitud}$)
Bits de página/desplazamiento	$\text{bits} = \log_2(\text{tamaño o cantidad correspondiente})$
Tamaño de tabla de páginas (1 nivel)	$N.^{\circ} \text{ de entradas} \times \text{tamaño de cada entrada}$
Buddy: tamaño de bloque asignado	La menor potencia de 2 que sea $\geq$ al tamaño pedido
Buddy: fusión (coalescencia)	Solo si el buddy verdadero está libre y juntos forman una potencia de 2 válida

Checklist de errores comunes a evitar

- Confundir 'fallo de página' (página no cargada, $V=0$) con 'error de direccionamiento' (dirección fuera de rango o página inexistente).
- Olvidar verificar el límite del segmento ANTES de sumar base+desplazamiento.
- En Buddy, asumir que dos bloques libres contiguos siempre se pueden fusionar — solo se puede si son 'buddies' verdaderos (formando una potencia de 2 válida que empiece en la dirección correcta).
- En particiones fijas vs. variables, no confundir fragmentación interna (sobra dentro de una partición ya asignada) con fragmentación externa (huecos libres demasiado chicos o no contiguos para usar).
- En paginación multinivel, recordar que la tabla de PRIMER nivel siempre se crea completa (entera), mientras que de los niveles inferiores solo se crean las tablas que realmente hacen falta según cuánta memoria usa el proceso.
- En los algoritmos de reemplazo, no mezclar el criterio de LRU (último uso) con el de LFU (frecuencia total de uso): son criterios distintos y pueden dar resultados distintos.
- Recordar que Óptimo y LRU nunca sufren la anomalía de Belady; FIFO sí puede sufrirla.

¡Éxitos en el examen! Si tenés dudas puntuales sobre algún ejercicio, repasá el bloque correspondiente y fijate en el razonamiento paso a paso, no solo en el resultado final.